

DLCA Lab 4: Simulator

Week 2

August 26, 2025

1 Introduction

In this week's lab, you shall be learning how to use a state-of-the art microarchitectural simulator (Champsim) to see the effects of different microarchitectural policies on different workloads.

2 Necessary tools

We shall be using make and g++ in order to compile C++ code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ make
make: *** No targets specified and no makefile found.  Stop.
$ g++
g++: fatal error: no input files
compilation terminated.
```

You will also need to ensure that your system has the traces installed, this can be checked by running:

```
$ ls $CS231_TRACE_DIR
bwaves.xz gcc.xz mcf.xz sssp.gz
```

You must also ensure that Champsim has been installed and setup in the Desktop folder of your lab machine, there must be a folder on your desktop called Champsim. You can ensure it is correct by running the following from your desktop:

```
ls Champsim/
bin branch btb champsim_config.json CITATION.cff config.sh
inc LICENSE Makefile obj prefetcher replacement run_champsim.sh src storage tracer
```

3 Structure of submission/templates

```
your-roll-no/
|- task1/
   |- cache.cc
   |- main.cc
   |- proper_plotter.py
```

| - ipc_vs_bypass.pdf

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

4 Tasks

This assignment consists of one subtasks. This task is expected to be submitted in-lab. It is intended to get you comfortable with modifying cache.cc and main.cc in Champsim. This is something you will need to do for your outlab assignment.

Some background

How to build Champsim?

To build champsim, all you have to do is run `./config.sh champsim_config.json && make`. This builds the champsim binary with the required cache management policy. You may modify the executable name by changing the `executable_name` inside `champsim_config.json` to something other than `bin/lru`.

How to run Champsim?

We have given you a tool to run Champsim, the tool takes in two arguments:

1. `TRACE_FILE`: This is the workload which is run on the simulator, you have been given four workloads which are found in the `$CS231_TRACE_DIR` folder
2. `BINARY`: This is the specific champsim binary which contains the simulator on which to run the workload

An example run of champsim looks like:

```
$ bash run_champsim.sh $CS231_TRACE_DIR/sssp.gz random
```

```
*** ChampSim Multicore Out-of-Order Simulator ***
...
```

Task 1: PC-based Discrimination in the LLC

It is a well known fact that most of the cache lines resident inside a Last-Level Cache are *dead-on-arrival*. This means, most of the lines inside the LLC get evicted before they receive any cache hits.

Hypothesis: *Cache lines brought in by the same Program Counter (PC), behave similarly.*

To test and possibly exploit this hypothesis, we shall be doing the following:

1. Throughout the warmup instructions, take note of how many LLC fills a PC causes and how many hits are received by cache lines filled by the PC
2. Once the warmup is over, per PC we have access to the *fill accuracy* of that PC, defined as

$$\text{fill accuracy}[PC] = \frac{\text{numHits}[PC]}{\text{numFills}[PC]}$$

3. Define a value, called the *bypass threshold* (BT). For an LLC fill, if the PC causing that fill has a *fill accuracy* $> BT$, we will fill it into the cache, otherwise we will not fill (*bypass* the fill)

You may be (rightly) asking yourself, what about LLC writebacks? They aren't caused by a particular PC, rather just by the event of an L2 dirty block eviction. Well let us assume that X is the dirty cache line that is being written back to LLC, let us assume that the PC that caused X to get filled is PC_X , we will consider PC_X as the PC causing the fill of X into the LLC.

Champsim source code can be very intimidating to work with, therefore, the TAs will be solving this assignment with you! We will assume a value of $BT = 0.2$ and solve the lab alongside you, explaining the various modifications that we need to make.

So what do you have to do?

Your job is to follow along with the TAs and understand and make the same changes. Finally, you have to use the proper `plotter.py` tool to make a plot of the *speedups* you got for various different values of BT . The instructions to use the tool are commented in the tool itself. We define $speedup[BT]$ as $speedup[b] = \frac{IPC_{BT=b}}{IPC_{no-bypass}}$

Your plot may end up looking like Figure 1:

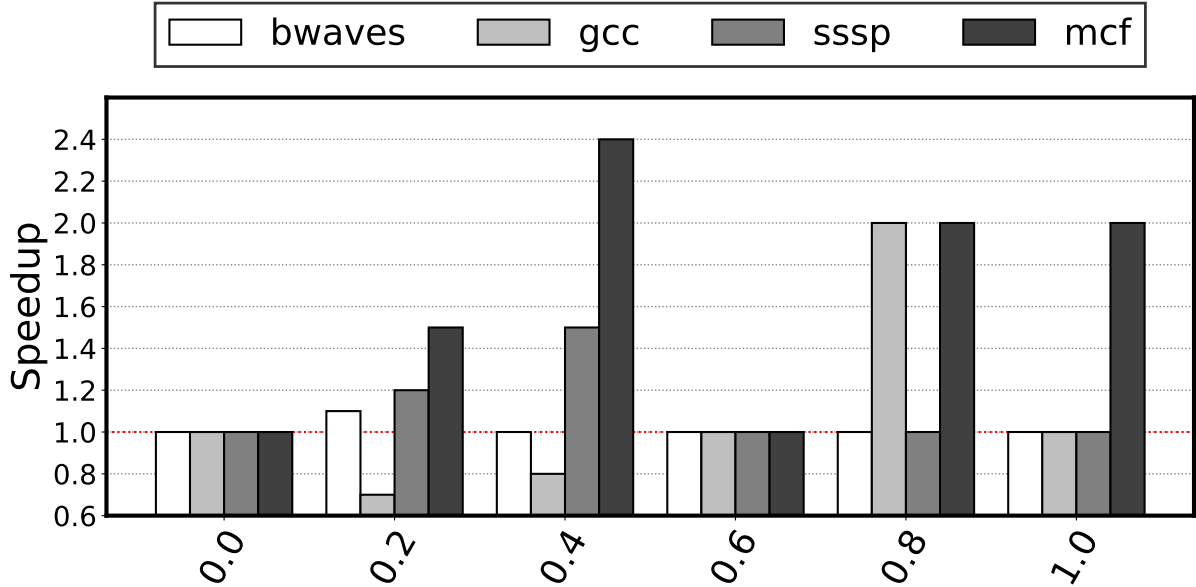


Figure 1: Speedup obtained as a function of BT , for four different workloads.

Note that the values shown in the figure are purely random, your values will differ.

The baseline IPCs for each trace are in Table 1

Trace	IPC
bwaves	0.353007
gcc	0.324587
mcf	0.453525
sssp	0.324201

Table 1: Traces and their IPC values in no-bypass configuration